

NBA In-Game Win Probability

Parallel Monte Carlo Simulation in SystemVerilog + Verilator

Project Report — July 21, 2026

1. Abstract

This project implements an FPGA-friendly SystemVerilog Monte Carlo engine that estimates each team's win probability from an in-progress NBA game state (score differential, time remaining, and possession). Sixteen parallel simulation cores, each driven by a Marsaglia xorshift32 RNG, replay possessions until the clock expires and aggregate thousands of trials under Verilator. An enhanced CLI selects live or scheduled games from ESPN's public scoreboard API and runs the simulator in one step. A historical backtest samples random mid-game moments from finished regular-season games and measures whether the model's favorite matched the final winner. This document records the architecture, implementation process, usage, and evaluation results.

2. Motivation and Problem

In-game win probability (WP) is widely used in sports analytics and broadcasts. Computing WP by Monte Carlo simulation of remaining possessions is a natural fit for hardware parallelism: each trial is independent, short, and RNG-heavy. The goal of this project is to express that algorithm as synthesizable SystemVerilog suitable for FPGA mapping, verify it with Verilator, and connect it to real NBA game state via a lightweight Python front-end — without paid sports data APIs or machine-learning frameworks.

3. Architecture

The design is layered from RNG → possession model → trial core → parallel aggregator → C++ testbench → Python CLI/backtest.

3.1 RTL Modules

Module	Role
xorshift32	Marsaglia xorshift32; full 32-bit step per cycle (13, 17, 5)
possession_outcome	Combinational map: RNG bits → points + duration
sim_core	FSM IDLE/SIM/FINISH; one game trial; unique CORE_ID seed
monte_carlo_top	16 cores, 4096 trials; aggregates wins_a / total_trials

3.2 Data Flow

```
CLI / backtest → game_state.txt → sim_main.cpp
→ monte_carlo_top (run, init_*, base_seed)
→ sim_core[i] → xorshift32 + possession_outcome
← done, total_trials, wins_a → printed report / metrics
```

Each core seeds with `base_seed XOR (CORE_ID * 0x9E3779B9)`. While run stays high, a finished core reloads the initial state and starts another trial without reloading the RNG, keeping consecutive trials decorrelated.

4. Possession Model

Outcome roll uses `rand_val[15:0]` (out of 65536). Duration is `8 + rand_val[19:16]` (uniform 8–23 seconds, ~15.5 average). Expected points per possession ≈ 1.16 , matching NBA averages.

Range	Pts	Prob	Interpretation
[0, 32768)	0	50%	Turnover / miss
[32768, 53740)	2	32%	Made 2 or two FTs
[53740, 62915)	3	14%	Made 3
[62915, 64226)	1	2%	Single FT
[64226, 65536)	4	2%	And-1 on a 3

5. How the Project Was Completed

Implementation followed a bottom-up RTL → software → validation path:

1. Created the `nba_montecarlo/` tree and `example_game_state.txt`.
2. Implemented `xorshift32`, `possession_outcome`, `sim_core`, and `monte_carlo_top` to the simulation FSM spec.
3. Wrote the Verilator C++ testbench (`sim_main.cpp`) and Makefile with `-std=c++14`.
4. Added ESPN scoreboard fetch (`fetch_game.py`) and later refactored shared helpers into `espn_client.py`.
5. Built an interactive CLI (`interactive_wp.py / make cli`) that selects a game and runs the sim.
6. Added historical backtest (`backtest_mcwp.py`) sampling mid-game ESPN play-by-play snapshots.
7. Generated this PDF via ReportLab from architecture notes and CSV results.

5.1 Project Layout

```
nba_montecarlo/  
■■■ rtl/           # SystemVerilog cores  
■■■ sim/           # Verilator C++ testbench  
■■■ scripts/       # CLI, ESPN client, backtest  
■■■ results/       # backtest_results.csv  
■■■ docs/          # this report generator + PDF  
■■■ Makefile  
■■■ game_state.txt
```

6. Interactive CLI Usage

From the project directory, run:

```
make cli
```

This fetches today's ESPN scoreboard, prefers live games (otherwise lists scheduled/final), prompts for possession (0 = home / Team A, 1 = away / Team B), writes `game_state.txt`, builds the simulator if needed, and prints win probabilities for both teams. Manual mode remains available by editing `game_state.txt` and running `make run`.

7. Backtest Methodology and Results

7.1 Method

The backtest does **not** train neural weights; it evaluates the fixed possession model. For each of N finished games (default 50) drawn from random dates in November 2024–March 2025:

- Fetch `scoreboard?dates=YYYYMMDD` and keep `state=post` finals.
- Fetch `summary?event=ID` and collect plays with scores and clocks.
- Sample one random play with regulation time remaining in (120, 2400] seconds (skip OT).
- Infer possession from the play's team id when possible; otherwise randomize and flag uncertainty.
- Run MCWP; let `p_home = wins_a / trials`.
- Count a correct guess when the favorite (`p_home ≥ 0.5`) matches the final winner.
- Report accuracy, mean absolute error, Brier score, and reliability bins.

Run with: `make backtest` (or `python3 scripts/backtest_mcwp.py --n 50`).

7.2 Results

Evaluated **50** mid-game snapshots. Favorite-match accuracy: **72.0%**. Mean $|p - \text{outcome}|$: **0.309**. Brier score: **0.152** (0 = perfect, 0.25 $\approx$ coin flip).

Away @ Home	Snap	t(s)	p_home	Final	OK
Toronto Rapt @ New Orleans	99-74	492	0.00	119-93	Y
Washington W @ Chicago Bull	63-75	1241	0.87	105-138	Y
Oklahoma Cit @ Golden State	97-96	167	0.35	105-101	Y
Milwaukee Bu @ Chicago Bull	72-72	1085	0.47	111-116	N
Boston Celti @ Chicago Bull	46-51	1784	0.66	138-129	N
Phoenix Suns @ Golden State	34-36	1995	0.54	105-109	Y
Sacramento K @ Boston Celti	84-78	622	0.21	114-97	Y
Indiana Pace @ Miami Heat	110-86	637	0.00	128-115	Y
LA Clippers @ Washington W	39-32	2005	0.29	121-96	Y
Brooklyn Net @ Indiana Pace	60-51	1189	0.22	99-105	N
New Orleans @ Philadelphia	110-97	287	0.01	123-115	Y
Memphis Griz @ Chicago Bull	62-50	1593	0.16	142-131	Y

(Showing up to 12 of 50 rows; full data in `results/backtest_results.csv`.)

8. Limitations and Future Work

Known limitations of the current model:

- No overtime modeling (ties resolved by RNG coin flip).
- No intentional-foul / end-of-game strategy.
- No team-specific offensive or defensive ratings; no home-court factor.
- Possession is often inferred imperfectly from ESPN play team ids.
- Thresholds were not auto-retuned from the backtest (evaluation only).

Suggested extensions: per-team points-per-possession parameters, home-court adjustment, momentum, foul-trouble modes, and optional threshold calibration against a larger historical sample.

— End of report —